Agent Memory Selection under Bounded Context

Which History Records Matter for Software-Engineering Agents?

Jorge Simão

Draft — September 2026

Abstract

Long-running language-model agents accumulate task statements, model actions, tool observations, source-code views, edits, tests, error messages, and intermediate hypotheses. The common baseline is to replay as much recent history as the model context permits, but recency alone ignores the causal and semantic roles of agent state. This paper studies agent-history selection as a separate problem from inference-engine caching: given a canonical trajectory, which records should remain active for the next model call?

We introduce an engine-independent record abstraction and evaluate bounded memory policies for software-engineering agents. The first harness is mini-swe-agent, whose linear message history, bash-only action interface, and minimal control loop make memory selection directly observable. We compare full history, token and turn tails, independently controlled head and tail turns, causal-bundle recency, a typed progress spine, lexical and dense retrieval over prior records, hybrid policies, and oracle utility selection. We separate record selection from within-record materialization and evaluate both frozen-trajectory counterfactuals and autonomous SWE-bench runs.

The primary outcomes are official task resolution, model calls, selected input tokens, repeated source acquisition, repeated verification, and first-divergence behavior. We additionally measure the age and semantic type of historical records that remain causally useful. The goal is not to optimize one serving engine, but to derive a portable agent-memory policy that can later be realized as text, persistent native K/V, or other backend state. We hypothesize that typed causal retention preserves agent quality more efficiently than token recency, and that its first benefit appears as equal task success with fewer calls, rereads, and retained tokens. In the first autonomous single-task pilot, however, same-version/span read supersession saves 6.43% of cumulative input tokens, changes the trajectory at its first exclusion, and ends in official failure. The primary full-history submission resolves only one of two executions, so this is evidence of a behavioral effect rather than an identified task-success penalty. On a second task identity, two new full-history controls again resolve one of two primary submissions despite identical declared generation settings and exact initial workspace identity. Post-hoc auxiliary grading of the two failed full-history submissions shows that both terminal workspaces contain resolving patches: the failures are in submission protocol rather than solution state. Calls and actions nevertheless remain unstable, so we stop the second task before any heuristic arm. The H3 terminal workspace also fails its auxiliary grade.

1 Introduction

Software-engineering agents differ from ordinary retrieval-augmented prompts in one crucial way: their context is not a static evidence set. It is a growing history of actions and consequences. A coding trajectory may contain the original task, source-code observations, edits, failed commands, test results, intermediate hypotheses, and final verification. Many of these records become irrelevant quickly, while others remain necessary long after they were created.

The usual history policy is recency. Given a context limit, an agent keeps the latest messages or tokens and discards older state. This is simple but ignores causal structure. An action and its observation form a unit. A source view may remain important after several later path-discovery commands. A mutation can remain relevant until a subsequent verification. A test result may be obsolete after another edit, while the task statement remains permanently relevant.

This paper isolates the memory-policy problem from the serving-engine problem. At model turn t, let the full canonical history be

$$H_t = [r_1, r_2, \ldots, r_n],$$

where each r_i is a logical agent record. A memory policy computes

$$S_t = \pi(H_t, Q_t, B),$$

where Q_t is the active task/state and B is a context budget. The selected plan S_t is then serialized to an ordinary model request. No native K/V mechanism is required for the main experiments.

This separation is deliberate. Prior PRA work studies retrieval, native-memory materialization, cross-document composition, and engine-specific K/V reuse. Those systems questions should not determine which semantic records an agent needs. Here we first identify useful logical memory policies. A separate runtime paper can later ask whether different engines realize the same selected plan correctly and efficiently.

We begin with mini-swe-agent because its control loop is minimal: a linear message history is sent to the model, the model emits shell actions, the environment executes them, and observations are appended to the same history. This minimizes hidden harness state and makes memory intervention easy to audit.

Research questions. We ask:

1. How much agent history can be removed before official task quality degrades?
2. Are causal-bundle and semantic-role policies more efficient than token or turn recency?
3. Which record types—source views, mutations, verification, progress, or task state—have the longest useful lifetimes?
4. Does bounded memory increase recovery behaviors such as rereading source or repeating tests?
5. How well do lexical, dense, and hybrid record retrieval approximate an oracle future-utility policy?
6. Do the best policies transfer to another agent harness and to persistent-native-memory execution?

Contributions planned. We target five contributions:

1. An engine-independent typed record model for agent trajectories.
2. A controlled policy ladder from token recency to causal, head–middle–tail, and role-aware retention.
3. A two-stage evaluation combining frozen-trajectory counterfactuals with autonomous SWE-bench runs.
4. Agent-specific efficiency metrics including avoidable rereads, repeated verification, and memory dependency horizon.

5. A frozen set of qualified logical policies that can later be realized by native-K/V runtimes without retuning the semantics.

2 Background and Motivation

2.1 Agent history differs from document retrieval

Document retrieval typically selects evidence that is externally stored and largely immutable. Agent memory contains generated and observed state whose meaning depends on the trajectory. A source observation may explain a later edit; an edit invalidates earlier verification; a rejected action may produce an error observation without becoming part of the accepted action history.

These dependencies motivate a record graph rather than an undifferentiated token stream.

2.2 Why mini-swe-agent

mini-swe-agent maintains a linear message list. At each step the model receives that list, returns one or more actions, the environment executes those actions, and formatted observations are appended. Because there is little additional memory management in the harness, the trajectory itself is the natural object for intervention.

We use mini-swe-agent as the first experimental harness, then test whether the highest-performing policies transfer to a richer open-source agent and to a record-native PRA agent.

mini-swe-agent exposes one generic Bash action interface. Consequently, tool choice is not a meaningful routing variable in this first study: the scientific variables are which action–observation history survives and how large tool observations are represented. We do not place a PRA gateway in the primary factorial. Gateway tool routing and result compaction are separate interventions that can be tested only after the logical memory policy is fixed.

3 Agent Record Model

We normalize trajectory messages into typed records:

$$\mathcal{R} = \{\textsc{Task}, \textsc{Action}, \textsc{Observation}, \textsc{Source}, \textsc{Mutation}, \\ \textsc{Verification}, \textsc{Progress}, \textsc{Error}, \textsc{Finalization}\}.$$

Each record contains a stable identifier, turn index, semantic type, causal group, source identity/spans where applicable, mutation target, and logical creation time.

3.1 Causal bundles

Actions and observations are grouped:

$$\textsc{Action}_i \leftrightarrow \textsc{Observation}_i.$$

Mutations and later verification can form a higher-level dependency:

$$\textsc{Mutation}_i \rightarrow \textsc{Verification}_j.$$

Policies may rank individual records, but required bundles are rounded up unless the experiment explicitly tests bundle-breaking.

4 Memory Selection Policies

4.1 Full history

FULL retains all canonical records and defines the reference trajectory.

4.2 Recency baselines

We test:

- **TOKEN-TAIL**: last B tokens;
- **TURN-TAIL**: last N complete turns;
- **RECORD-TAIL**: last M records;
- **CAUSAL-TAIL**: last N complete causal bundles.

4.3 Independent head, middle, and tail

Recency and early-run retention are separate variables. We therefore define a three-zone policy

$$S_t = I \cup H_h \cup \pi_{\text{middle}}(M_t, Q_t, B') \cup T_\tau,$$

where I is the immutable system prompt and task statement, H_h contains the first h complete action–observation turns, T_τ contains the latest τ complete turns, and the selector operates only on the non-overlapping middle M_t. The head and tail controls are independent; $h = 0$ never removes I. Our pilot grid uses

$$h \in \{0, 1, 2, 4\}, \qquad \tau \in \{1, 2, 3, 5, 10\}.$$

Frozen replay eliminates dominated combinations before autonomous evaluation. This factorization tests whether early localization or plan formation remains useful after leaving a conventional recency window.

Head and tail turns remain causally complete. An oversized tool observation is the only permitted within-turn exception: its action, return code, resource identity, mutation/verification status, error headline, and causal identity remain present while its body may use a separately recorded detail policy. When immutable/head/tail state exceeds the requested budget, we report mandatory overflow and realized retention rather than silently breaking a turn.

4.4 Typed progress spine

The structured policy retains

$$S_{\text{spine}} = T \cup R_{\text{recent}} \cup S_{\text{source}} \cup M_{\text{mutation}} \cup V_{\text{verification}} \cup P_{\text{progress}},$$

where each component denotes the latest required records of that semantic role.

4.5 Retrieval policies

We compare lexical, dense, and hybrid retrieval over historical records using a query formed from

$$\text{task} + \text{current progress} + \text{latest active observation}.$$

The hybrid policy combines a mandatory structured spine with retrieved records.

4.6 Resource/effect DAG and negative exclusion

Positive selection asks which records are likely useful. We additionally test the dual question: which records are operationally obsolete under a declared resource-state abstraction? We construct an acyclic graph over actions, observations, immutable resource versions, and precisely defined projections. Typed edges include Returns, ReadsFrom, Writes, SupersededBy, and SubsumedBy. Effects carry provenance from declared tool semantics, runtime tracing, static heuristics, or Unknown; unknown effects prohibit certified exclusion.

The two-stage policy is

$$H_t \xrightarrow{\text{certified exclusion}} H'_t \xrightarrow[\text{only if still over } B]{\pi_{\text{positive}}} S_t.$$

A certificate names the rule, witnesses, proof scope, resource-version fingerprints, cwd/environment identity, completeness conditions, and explicit assumptions. We separate three tiers: Dag-Certified, Dag-Recoverable, and Dag-Heuristic. Only the first is removed by the conservative $B = 100\%$ exclusion arm; the others require the same counterfactual and autonomous qualification as positive retrieval.

The scope of "certified" is deliberately narrow. A later record can be proven to supersede an older current-state projection, but this does not prove that the older text contains no decision-relevant diagnostic fact. Moreover, deleting even byte-identical repeated text changes token positions and attention. Thus the claimed property is *certificate-backed operational obsolescence*, not risk-free behavioral equivalence. Behavioral preservation remains an empirical outcome measured by frozen replay and autonomous tasks.

The initial executable rule, Trace-Exact-Operation-Result, compares the executed command and complete observation rather than the assistant's private reasoning prose. It fires only when cwd, environment identity, affected resource-version fingerprints, return code, and observation bytes agree under runtime-traced or declared-complete semantics. The older whole causal bundle is then operationally redundant. Different reasoning prose is intentionally outside the certificate, which is why frozen behavioral qualification remains required even for this strongest exclusion tier.

Initial rules fail closed: system/task/current state and unresolved errors are protected; arbitrary or untraced Bash is Unknown; output must be complete and non-truncated; file reads require canonical paths and version/span identity; git queries require separate HEAD, index, and worktree identities; and action–observation bundles remain intact. A write invalidates an old record as an authority on current state but does not by itself authorize semantic deletion. Similarly, a later passing test does not automatically erase a prior failure's diagnostic information.

4.6.1 Auditable negative-selection heuristic ladder

Certification coverage will initially be sparse, so we test four explicitly non-certified negative-selection families before invoking learned relevance. All retire complete action–observation groups, preserve independent head and tail floors, and emit a compact inactive tombstone containing the rule, resource, and witness identities. The tombstone is controller provenance and is not serialized into the model request; serializing it would be a distinct summarization treatment.

H1–Consumed Search retires a discovery operation such as `find`, `fd`, `rg --files`, or filename-only `grep -l` after a returned resource is consumed by a concrete source read and the trajectory subsequently advances to a non-read operation. Source-bearing `grep -n` and ordinary `rg` matches count as reads, not discovery. The delay $K_f \in \{0, 1, 2, 4\}$ starts at that transition and measures whether the location evidence retains short-lived value after consumption. An unresolved search/read exploration chain is not retired.

H2–Write Consolidation is split because absence of a rewrite does not imply that a mutation was good. H2a requires an observed resource change and a later read or diff of the same post-write version. H2b requires a successful verification with an explicit dependency on the written resource; a workspace-wide passing check is a separately labelled relaxed arm. Only the raw write bundle is retired, while the later current-state or verification evidence remains. The bare "no later rewrite" rule is retained solely as an aggressive ablation with $K_w \in \{1, 2, 4\}$.

H3–Read Supersession retains the latest K_r observations, for $K_r \in \{1, 2, 3\}$, covering each resource–version–span key. A whole-file read covers a line span; a later disjoint `sed` region does not; grep/tail projections require the same query signature. Missing versions use a no-intervening-write epoch only in the heuristic tier and never become a certificate.

H4–Bounded Working Set retains the latest $K_x \in \{2, 3, 4, 6\}$ distinct read resources while pinning resources named by the task, mutated state, unresolved failures, the current tail, and declared dependencies. Cross-file software dependencies make H4 the riskiest family; it is reported as a capacity heuristic rather than safe exclusion.

The predeclared sequence isolates H1, H2a, H2b, H3, and H4 before combining H1+H3, H2+H3, H1+H2+H3, and finally all four. If the negative stage remains over budget, the original absolute budget is passed to the positive selector. This ordering identifies whether failure came from a particular retirement rule or from subsequent relevance selection.

4.6.2 Tool-semantics ownership and runtime handoff

mini-swe-agent exposes only one generic Bash tool, so the Paper 8.5 harness provides a Bash-specific semantics adapter that classifies common commands and, when available, augments them with runtime filesystem/version tracing. This special case belongs in the harness-facing adapter, not in a PRA inference engine. The first implementation certifies only simple allowlisted read or pure commands. Pipelines, lists, substitutions, redirections, heredocs, multiline scripts, timeouts, and truncated observations fail closed even when a coarse regex suggests a familiar command category.

The instrumented Docker environment leaves the model-visible observation text unchanged and attaches a versioned metadata envelope containing canonical cwd, environment identity, return and completeness status, pre/post resource fingerprints, and pre/post repository-state fingerprints. Before every action it also stores the base commit, separate binary index and worktree patches, and an archive of nonignored untracked files. Untracked file contents, not only their names, participate in the state digest. This is an exact, restorable repository state under the declared scope; ignored artifacts, container-global state, network state, and resources outside the repository remain explicitly out of scope. A restore verifier refuses incomplete receipts, dirty targets, digest mismatches, and unsafe archive paths.

The portable interface is instead a tool-semantics provider. A developer or agent SDK can declare generic reads, writes, deletes, verification targets, resource versions, causal dependencies, output completeness, and confidence. It may also attach record directives such as protected, certified-excludable, or preferred-for-selection, with a certificate and witness IDs. Future tool categories implement the same effect vocabulary without teaching the engine about Bash, Excel, browsers, databases, or particular agent frameworks.

When the policy returns to Paper 4.5, the PRA runtime consumes only a frozen logical plan and generic record metadata. It validates identities, dependencies, and lifecycle constraints and realizes the chosen records physically; it does not infer application-specific semantics or rerun the DAG policy inside each engine. This preserves one agent-independent engine contract while allowing a semantically richer harness or SDK to make selection decisions.

4.7 Oracle

A retrospective oracle estimates future usefulness through record/bundle ablation on successful trajectories. It is an upper bound and is never used as a deployment policy.

5 Materialization Policies

Record selection and record detail are separate variables.

For selected records, we compare:

- full record;
- matched source span;
- bounded head/tail;
- hierarchical child span;
- optional summary;
- compact provenance stub for an excluded record.

The first experiments use whole-record materialization so that selector effects remain identifiable. A targeted second factorial applies detail reduction only to oversized tool observations. Source views retain matched symbol/file spans; test outputs retain the command, return code, failed-test identities, exception, and relevant stack frames; search results retain matched lines and locations; mutations retain their changed span or diff. This reuses gateway-style compaction semantics without introducing a gateway or an engine into the study.

Negative exclusion therefore has two separately measured realizations. The primary arm removes the retired payload from the model request while retaining its tombstone only in controller state. A second *model-visible stub* arm replaces it with compact metadata such as resource, mutation/verification status, and superseding-witness identity. The latter may preserve useful provenance, but it is a materialization/compression treatment rather than pure exclusion and its stub tokens count against the realized budget. This cleanly separates binary HOT→WARM retirement from HOT→STUB presentation.

6 Experimental Design

6.1 Phase I: frozen-trajectory screening

We begin from successful full-history trajectories. At each historical model call we restore the corresponding workspace, apply a candidate memory policy, and query the same frozen model.

We measure:

- exact next-response agreement;
- exact action/command agreement;
- action equivalence;
- target-file agreement;

- first-divergence turn;
- retained tokens and records.

Frozen screening permits broad policy and budget sweeps without paying for full autonomous reruns.

The first frozen cohort follows a gate rather than a Cartesian sweep. On the same instrumented successful trajectory, frozen model revision, tokenizer, temperature zero, generation parameters, and decision snapshots, we run:

1. FULL, establishing contemporaneous reproducibility;
2. DAG-EXCLUDE at a nominal 100% ceiling;
3. a recency/token-tail control matched to the DAG arm's realized token count at each decision;
4. head–middle–tail policies at a 90% minimum-retention floor, rounded upward at whole causal-turn boundaries.

Failure of FULL stops interpretation of later arms because it means the frozen baseline itself was not reproduced. Autonomous qualification and oracle add-back tests begin only after this ordered next-action gate. The matched-tail control distinguishes the value of certified semantic exclusion from the value of merely removing the same number of old tokens.

The two budget contracts are intentionally different. The matched-tail arm must not exceed the DAG arm's realized token count, so it rounds down and reports unused capacity. A treatment named "90%", however, must retain at least $\lceil 0.9N \rceil$ logical tokens: it follows the policy ranking until the next indivisible causal bundle crosses that floor and reports the resulting whole-turn overshoot. A preliminary implementation treated 90% as a hard ceiling and retained only 71.9% on one decision; that partial run is quarantined as a budget-contract failure rather than interpreted as policy quality.

The repository state is part of the frozen decision point. Replaying turn t restores the exact pre-action repository state within the declared checkpoint scope, not merely the textual history, so a candidate command is evaluated against the same tracked, staged, worktree, and nonignored-untracked contents as the reference action.

6.2 Phase II: autonomous evaluation

Policies that survive frozen screening are run autonomously from a fresh task sandbox. Final success is measured by the official SWE-bench grader.

Autonomous evaluation is required because a divergent action may still recover later, while a seemingly small memory omission may cause many additional calls.

6.3 Benchmarks

We begin with the existing SWE-bench Verified Easy-14 task sequence, then expand to Easy-50 and a broader Verified sample.

Easy-14 is a mechanism/debug cohort. Publication-level claims require a larger task set.

6.4 Budgets

We evaluate relative budgets

$$100,\ 90,\ 75,\ 50,\ 25\%$$

and absolute budgets

$$2\text{K},\ 4\text{K},\ 8\text{K},\ 16\text{K},\ 32\text{K}.$$

Every request records both requested and realized retention.

7 Metrics

7.1 Task outcome

Primary:

$$\text{SWE-bench resolved.}$$

Additional metrics are model calls to completion, shell actions, and final patch/test status.

7.2 Memory efficiency

We record full-history tokens, selected tokens, retention fraction, selected record count, selected records by semantic role, and the largest retained record.

For DAG policies we additionally report certified, recoverable, heuristic, and unknown-effect tokens separately; certificate coverage and abstention; records removed per rule; and certificate violations under frozen add-back tests. A certificate can be operationally sound while the LLM still changes behavior, so operational certificate validity and next-action survival are distinct metrics.

7.3 Recovery behavior

A memory policy may preserve final task success by forcing the agent to reacquire forgotten state. We therefore count repeated source reads, overlapping source rereads, repeated searches for the same fact, repeated tests without relevant intervening mutation, and repeated failed edits. We call this *reacquisition overhead*.

For every negative arm we also report exclusions and tokens by rule, the resource and witness provenance stored in inactive tombstones, and the fraction of exclusions whose information is later reacquired. Frozen replay includes a narrow immediate false-exclusion proxy: the selected-history generation reads a hidden resource when contemporaneous FULL does not. This proxy is not a causal error rate—the primary evidence remains autonomous reacquisition, calls-to-solution, and official task success.

7.4 Dependency horizon

For a useful historical record r_i consumed at turn t, define

$$d(r_i, t) = t - \text{created}(r_i).$$

We report dependency-age distributions by record type. This estimates how long coding-agent memory must remain accessible.

8 Planned Policy Matrix

Table 1: Primary selector-by-budget matrix. Each cell reports official task success, calls/task, selected tokens, and avoidable rereads.

Policy	100%	90%	75%	50%	25%
FULL	–	–	–	–	–
Token tail	TBD	TBD	TBD	TBD	TBD
Turn tail	TBD	TBD	TBD	TBD	TBD
Causal tail	TBD	TBD	TBD	TBD	TBD
Progress spine	TBD	TBD	TBD	TBD	TBD
Lexical	TBD	TBD	TBD	TBD	TBD
Dense	TBD	TBD	TBD	TBD	TBD
Hybrid	TBD	TBD	TBD	TBD	TBD
DAG-certified exclusion	TBD	TBD	TBD	TBD	TBD
DAG-certified + hybrid	TBD	TBD	TBD	TBD	TBD
Oracle	TBD	TBD	TBD	TBD	TBD

Within each structured-policy row, we first screen the independent head–tail grid. The selected middle policy is then compared at matched materialized token counts. This prevents a large mandatory head from being mistaken for superior middle retrieval and prevents percentage budgets from hiding mandatory round-up.

8.1 Pilot structural screen

Before issuing counterfactual model calls, we ran the recordizer and selector over three successful mini-swe-agent reference trajectories containing 69 historical model decisions. The full grid contains five budget fractions, four head floors, five tail floors, five initial middle policies, and two DAG-first compositions, for 48,300 selection decisions and 2,100 aggregated cells. This screen uses a whitespace counter and is therefore evidence only about structural feasibility, not model token counts or policy quality.

Table 2 shows one illustrative slice with one first turn and two latest turns retained. Whole causal bundles leave some budget unused. Conversely, immutable and mandatory state can exceed the ceiling. At a 90% request the basic head–tail policy overflows on 16 of 69 decisions, while the role-aware progress spine overflows on 22 of 69. These are expected round-up outcomes, concentrated when the trajectory is short; they demonstrate why requested percentage, realized retention, and mandatory overflow must be reported separately.

The legacy trajectories lack cwd, environment, output-completeness, and resource-version fingerprints. The certificate-backed DAG arm therefore correctly abstains and retains 100% at $B = 100\%$. Static Bash analysis exposes 458 superseded-current-state and 75 write-invalidated-state candidate occurrences across the 69 growing prefixes, but no trace-exact duplicate and no certified exclusion. These are repeated candidate occurrences, not unique records, and are not removed. This is an instrumentation requirement, not a negative quality result.

Table 2: Aggregate realized retention in the structural-only pilot for $h = 1, \tau = 2$. These values do not measure next-action or task quality.

Policy	90%	75%	50%	25%
Middle none	55.5%	55.5%	55.5%	55.5%
Progress spine	66.5%	66.5%	66.5%	66.5%
Middle recency	85.9%	76.7%	59.6%	55.5%
Middle lexical	89.5%	77.1%	59.6%	55.5%
Spine + lexical	91.6%	80.0%	67.4%	66.5%

We also ran a structural-only opportunity screen for H1–H4 over the 23 growing prefixes of the first instrumented trajectory, with $h = 0$, two tail turns pinned, and a whitespace counter. After making explicit truncation receipts fail closed, H1 with $K_f = 0$ retires 938 cumulative whitespace tokens (1.0%); delaying to $K_f = 4$ reduces this to 670 (0.7%). Version-guarded H2a retires 308 (0.3%), and H3 with $K_r = 1$ retires 177 (0.2%); $K_r \geq 2$ finds no opportunity on this trace. Guarded H2b correctly abstains because the captured verification lacks an explicit resource dependency. H4 with $K_x = 2$ has the largest opportunity, 3,180 tokens (3.5%), while $K_x \geq 3$ abstains after dependency pins. H1+H3 removes 1,115 (1.2%), and all guarded rules plus H4 at $K_x = 4$ remove 1,423 (1.6%). These are cumulative prefix occurrences rather than unique records, and they measure neither tokenizer-exact saving, next-action quality, nor task success. Their purpose is to confirm that the arms fire differently before spending model calls; the frozen gate and first autonomous pilot follow below.

Repeating the screen with the pinned Qwen3-Coder tokenizer changes the counts but not the ordering. Across 199,109 cumulative full-history tokens, H1 at $K_f = 0$ removes 2,660 (1.34%), H2a removes 833 (0.42%), H3 at $K_r = 1$ removes 603 (0.30%), and guarded H2b abstains. Aggressive H4 at $K_x = 2$ removes 11,370 (5.71%), whereas $K_x \geq 3$ abstains under the dependency pins. These are still opportunity counts, not behavior results.

8.2 First instrumented frozen cohort

We then executed the ordered frozen gate on `scikit-learn__scikit-learn-13135`. The reference is a successful 23-decision ordinary-text mini-swe-agent trajectory using `qwen3-coder:30b`, its pinned tokenizer, temperature zero, and restored decision-point workspaces. This is next-action evidence, not yet an autonomous policy success rate. Table 3 compares each arm directly with a contemporaneous seeded FULL replay. Temperature zero alone was insufficient: an unseeded repeat diverged at decision 3 despite identical history through decision 20, so the original unseeded policy matrix is quarantined rather than interpreted as selection evidence. With seed zero, independent FULL-A and FULL-B runs match all 23 generated contents byte-for-byte. Exact-command and conservative-action rates use the 22 decisions on which FULL emitted a command; FULL's decision 22 is itself format-invalid.

Table 3: First instrumented frozen cohort. Saving is cumulative over growing decision prefixes. "Final" means equality of the frozen decision-23 action, not autonomous SWE-bench resolution.

Treatment	Mean ret.	Saving	Exact text	Exact cmd.	Cons. action	First text div.	Final
FULL	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
FULL repeat	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
DAG-EXCLUDE@100	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
Matched token tail	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
H1 consumed search, $K_f = 0$	98.9%	1.3%	52.2%	59.1%	68.2%	10	yes
H1 consumed search, $K_f = 4$	99.2%	1.0%	65.2%	68.2%	72.7%	14	yes
H2a current-state read	99.7%	0.4%	73.9%	77.3%	77.3%	17	yes
H2b verified write	100.0%	0.0%	100.0%	100.0%	100.0%	–	yes
H3 same span, $K_r = 1$	99.8%	0.3%	91.3%	95.5%	95.5%	21	yes
H4 working set, $K_x = 2$	95.1%	5.7%	43.5%	54.5%	59.1%	9	yes
H1+H3	98.6%	1.6%	47.8%	54.5%	59.1%	10	yes
H2a+H3	99.4%	0.7%	73.9%	77.3%	77.3%	17	yes
All guarded, $K_x = 4$	98.3%	2.1%	52.2%	59.1%	63.6%	10	yes

Two mechanism findings are currently admissible. First, the instrumented trace supports one TRACE-EXACT-OPERATION-RESULT certificate: two complete reads execute the same command and return the same bytes for the same cwd, environment, and resource version despite different reasoning prose. Removing the older turn saves only 603 cumulative tokens. It changes decisions 21–22, from writing a temporary diff to displaying it and then another invalid response, but returns to the exact final submission action at decision 23. Operational proof and behavioral identity are therefore empirically distinct even in the strongest current DAG tier.

The new within-observation token-tail control materially matches the DAG arm: 198,500 versus 198,506 cumulative materialized tokens. Both first change the action at decision 21, preserve 95.5% conservative action agreement, and return to the exact final action. This one trace therefore does not yet show a quality advantage for certified exclusion over recency at equal materialized context.

Second, immediate H1 is too aggressive. Once a discovery result is consumed by a concrete read and followed by a non-read transition, it removes one 190-token causal turn from each later prefix. The first text change occurs at decision 10, the first command-string change at decision 12, and the first conservatively different shell action at decision 13. Across all prefixes it saves 2,660 tokens (1.34%) but preserves only 68.2% conservative action agreement, with six immediate reacquisitions and one policy-excess reacquisition. A search listing can remain a durable navigation map after one file is opened. Delaying retirement to $K_f = 4$ moves first conservative action divergence from decision 13 to 16 and raises action agreement only to 72.7%, while five immediate reacquisitions remain. Delay is therefore insufficient: the next H1 refinement must protect unresolved discovery branches or retain a compact resource-map tombstone.

We implemented the strict unresolved-branch variant separately. It requires a complete successful read/diff for every concrete resource returned by the search, followed by the transition and K_f delay. It abstains at every prefix for all $K_f \in \{0, 1, 2, 4\}$ on this trace because several discovered example and test-file branches were never consumed. This is the correct conservative outcome, but supplies no saving; a compact model-visible resource-map stub is the next discriminating treatment.

H2a also fails as a safe-exclusion claim. Retiring a 119-token successful-write turn after a complete same-version read/diff saves 833 cumulative tokens (0.42%), but first changes the shell action at decision 17, preserves only 77.3% conservative action agreement, and changes action validity at decision 20. The current bytes represent file state, but they do not necessarily carry the mutation's intent or the agent's progress state. A model-visible mutation tombstone and the stricter verified-write H2b arm must therefore be evaluated separately. H2b correctly abstains on this trace because the verification record lacks explicit dependency metadata; its 23 outputs exactly match

FULL. This is fail-closed correctness, not evidence that H2b can save context when a verification receipt is available.

On this trace H3 at $K_r = 1$ retires exactly the same older read as the trace-exact DAG certificate, so all selected histories and generated outcomes coincide: 603 cumulative tokens saved and 95.5% conservative action agreement. This does not establish the safety of H3's weaker version/span criterion; a task containing non-identical overlapping reads is required to separate it from byte-identical duplicate elimination.

H4 at $K_x = 2$ reaches the largest saving, 11,370 cumulative tokens (5.71%), but changes the action as soon as it first fires at decision 9 and preserves only 59.1% conservative action agreement. Its zero immediate-reacquisition count is not exculpatory: frozen decisions do not compound, and the generated action can change without explicitly rereading the hidden resource. H4 remains an aggressive capacity baseline, not a safe rule.

The two-rule combinations show no positive interaction on this trace. H1+H3 saves 3,263 cumulative tokens (1.64%) but falls to 59.1% conservative action agreement, below H1 alone. H2a+H3 saves 1,436 (0.72%) and retains H2a's 77.3% action agreement because H2a has already changed the late decisions on which H3 fires. The predeclared three-rule H1+H3+H2b arm is input-equivalent to H1+H3 here because H2b abstains; it is recorded as a deterministic equivalence, not charged as another model cohort.

Finally, the all-guarded treatment at $K_x = 4$ saves 4,096 cumulative tokens (2.06%) with 63.6% conservative action agreement. H2b and H4 abstain at that setting, so this is effectively H1+H2a+H3. Its slight recovery relative to H1+H3 despite removing more context demonstrates non-monotone model sensitivity, not a robust policy improvement.

8.3 First autonomous qualification

We next ran an autonomous pilot on `django__django-15277` from fresh SWE-bench workspaces. All executions use mini-swe-agent 2.4.6, `qwen3-coder:30b`, the pinned Qwen tokenizer, temperature zero, top-$p = 1$, seed zero, whole-record text materialization, and the official SWE-bench Docker grader. Earlier pilot attempts that were no-ops or otherwise invalid were quarantined and do not enter Table 4. The corrected proxy forwards the incoming request exactly while a negative policy is a logical no-op; the included H3 run records five such exact decisions before its first exclusion. The primary endpoint is the official grade of the model patch submitted through the normal agent protocol. We additionally report a post-hoc official grade of a provenance-checked patch extracted from terminal tracked-workspace state. This auxiliary outcome diagnoses solution state; it never replaces or relabels the primary submission outcome. Extraction fails closed if untracked state makes a tracked-only patch incomplete.

Table 4: First autonomous single-task pilot. Token totals are cumulative message-content tokens and exclude chat-template tokens. Because the rollouts diverge, their full-history totals and call counts are not paired request prefixes. H3's lower call count is failure termination, not fewer calls to solution. "Aux." is the separately labelled final-workspace outcome.

Policy	Primary	Aux.	Calls	Full tok.	Selected tok.	Saving	Reacq.
FULL-A	1/1	unavailable	26	131,530	131,530	0.00%	0
FULL-B	0/1	1/1	29	154,173	154,173	0.00%	0
H3, $K_r = 1$	0/1	0/1	19	91,520	85,640	6.43%	6

FULL-A passes the primary official grader. Its auxiliary extraction is unavailable because the terminal checkpoint includes untracked `patch.txt`; rejecting an incomplete tracked-only reconstruc-

tion is the required fail-closed outcome. FULL-B makes the intended code change but diverges from FULL-A at call 9 and finally prints a source excerpt rather than a unified diff, so its primary submission is rejected. The provenance-checked 749-byte auxiliary workspace patch (SHA-256 prefix `01522b`) resolves officially. Thus FULL-B's primary failure is submission protocol, not solution state.

The primary FULL submission endpoint remains 1/2 on this task, while the available primary and auxiliary evidence establishes resolving code states for both executions. The trajectories still vary materially in calls and actions. One H3 failure cannot estimate a population success-rate delta. A separate fixed first-request probe reproduces assistant content and command exactly in 3/3 calls; this does not qualify a growing autonomous trajectory, where generated reasoning and environment observations enter later requests.

The qualified H3 rollout has the same model-visible request as FULL-B through call 5. Call 6 is the first actual exclusion: assistant reasoning changes immediately but its shell command remains the same. The shell command first changes at call 7. This is a causal next-action divergence, not by itself proof that selection caused the eventual official failure. The subsequent trajectory records six reacquisitions of excluded resources. It eventually applies an unscoped global `sed` replacement: the intended guard appears in `CharField`, but the already guarded `BinaryField` is also rewritten, producing a malformed duplicate `if`. The final submission command prints broad `grep` context rather than a Git diff; consequently, the submitted `model_patch` is not a diff and the primary grader reports a patch-application error. Unlike FULL-B, the H3 workspace is not a resolving solution: its 1,090-byte auxiliary patch (SHA-256 prefix `b1171e`) contains the malformed duplicate guard and also scores 0/1.

The scientific conclusion is deliberately narrow. H3's same-version/span condition is not behaviorally inert: a newer source read can supersede exposed bytes without proving the older record's localization, reasoning, or mutation intent irrelevant. The model-visible H3 trajectory and failure mechanism were reproduced exactly in a second execution after the no-op request fix, but only the corrected execution enters the table. The auxiliary comparison separates the observed failure mechanisms: FULL-B reaches a resolving state but violates the submission protocol, whereas H3 fails under both outcome definitions. This single task still does not estimate a population H3 penalty. Frozen replay remains a screening tool, while autonomous claims require repeated FULL controls and multiple task identities. H3 is explicitly a heuristic arm; certificate-backed and strict exclusions continue to fail closed when their evidence predicates are not met. The next H3 treatment should preserve a compact model-visible source/progress tombstone or require stronger evidence than shared resource-version and span.

8.4 FULL-control replication on a second task

We repeated the two-control procedure on `django__django-15368`. The two executions use the same pinned model, tokenizer and decoding fields as Task 01, fresh containers from the same image, and identical initial workspace and environment fingerprints. All 43 model requests are exact FULL pass-through and no request has an upstream error. Table 5 summarizes the result.

Table 5: Autonomous FULL-control replication across task identities. P is the primary submission outcome and A is the auxiliary final-workspace outcome. Calls are shown in parentheses. First command divergence compares the two FULL controls within a task; it is not a policy divergence.

Task	A:P	A:A	B:P	B:A	First cmd. div.	Heuristic
Task 01	1/1 (26)	unavailable	0/1 (29)	1/1	9	H3 pilot
Task 02	1/1 (30)	unavailable	0/1 (13)	1/1	6	none

Task 02 FULL-A resolves officially. FULL-B submits a source excerpt instead of a unified diff and receives a primary official patch-application error. The provenance-checked 731-byte auxiliary workspace patch (SHA-256 prefix `f5d867`) resolves 1/1. As on Task 01, FULL-B therefore reaches a resolving solution state but fails the submission protocol. FULL-A's auxiliary extraction is unavailable because untracked `bulk_update_fix.patch` and `test_fix.py` make a tracked-only reconstruction incomplete.

The Task 02 request sequence is identical through call 5. On that identical call-5 request, the assistant content differs while the Bash command remains equal; the accumulated request and command first differ at call 6. The auxiliary success narrows the failure to submission behavior, but does not remove the material instability in reasoning, calls, and actions. Temperature zero and a fixed seed cannot be treated as a guarantee of repeatable generation or agent protocol adherence for this endpoint.

The predeclared Task 02 gate required both FULL controls to resolve with sufficiently stable trajectories before testing strict H1. That primary gate fails, so we run no Task 02 H1, H2a, H3, or combined policy. Across two task identities, the primary FULL submission resolves exactly one of two controls on each, while auxiliary grading shows that both FULL-B workspaces resolve. This replicated protocol and trajectory instability strengthens the need for repeated FULL controls, but four runs are not a population estimate of model or agent reliability.

The auditable artifacts are under `docs/papers/shared/results/paper8_5_agent_memory/autonomous_task01_qwen30` and `autonomous_task02_qwen30`; they include per-request selection receipts, full trajectories, token counters, primary submission outcomes, separately labelled auxiliary workspace outcomes, and frozen run provenance.

9 Semantic-Role Ablations

For the typed progress spine, we remove one role at a time:

$$-\textsc{Source}, \quad -\textsc{Mutation}, \quad -\textsc{Verification}, \quad -\textsc{Progress}, \quad -\textsc{Recent}.$$

We measure official success, calls, rereads, repeated tests, and first-divergence rate. These ablations test which semantic roles carry long-lived task state.

10 Oracle Utility

For a causal bundle g and future decision point t, define

$$U(g,t) = \text{Quality}(H_t) - \text{Quality}(H_t \setminus g).$$

The first implementation uses action-level behavioral outcomes rather than token-level attribution. Its future reference action is available only to the offline scorer and is never serialized into model input.

Bundles are analyzed in the hierarchy

$$\text{record type} \rightarrow \text{causal bundle} \rightarrow \text{source span}.$$

The oracle provides an upper bound on how compact agent history could be if future utility were known.

We distinguish two oracle levels. The *next-action oracle* fixes system/task/head/tail state and searches subsets of middle causal bundles, scoring exact command, semantic command equivalence,

target file, and edit intent. Leave-one-bundle-out and role ablations provide the first utility labels; pairwise ablations and bounded beam or branch-and-bound search address non-additive bundle interactions before a subset is called minimal. The more expensive *rollout oracle* resumes a small predeclared set of decision points from exact workspace snapshots and scores eventual official success, calls, and recovery behavior. It tests whether next-action agreement is an adequate proxy for task utility.

We also express the oracle in the dual exclusion form. Holding protected state fixed, it searches for the largest set of middle bundles whose removal preserves the scoring outcome. This upper bound is compared with DAG-certified exclusions using behavioral precision (how often a certified exclusion survives the counterfactual), removal recall relative to the oracle-removable set, and token coverage. "Never used again" in the observed path is not an oracle label; utility must be established by counterfactual generation or rollout.

11 Transfer

11.1 Second open-source harness

After policy qualification on mini-swe-agent, we transfer only the top policies to one richer harness such as SWE-agent or OpenHands. The goal is not a second exhaustive grid, but to test whether semantic-role findings generalize when the harness has richer tools and memory processing.

11.2 PRA Agent

The final transfer uses PRA Agent, where typed task, tool, result, progress, and subagent records are native objects. This tests whether a record-native agent can exploit the same policy more cleanly than a retrofitted linear-message harness.

12 Handoff to Native Runtime Evaluation

Paper 8.5 should freeze a small set of logical policies, for example

$$\{\textsc{Full}, \textsc{Balanced}, \textsc{Aggressive}\}.$$

A runtime study can then map the exact same `AgentMemoryPlan` to text replay, HF cache views, MLX interval views, vLLM pages, llama.cpp resident slots, and SGLang cache objects. The runtime study must not retune selection semantics per engine.

Paper 8.5 reports selected logical tokens, not K/V savings. K/V residence, copying, re-encoding, TTFT, and latency are measured only after the frozen plan is transferred to the runtime study.

13 Expected Outcomes

Our main hypothesis is

$$\text{token recency} < \text{causal recency} < \text{typed progress spine} + \text{retrieval}$$

at matched context budgets.

A positive result need not initially increase raw task success. A practically important result is

same task success + fewer model calls + fewer rereads + fewer repeated tests + less retained history.

A negative result would also be informative: if simple recency performs as well as structured policies, coding-agent memory may be more local than assumed.

14 Limitations

Text realization recontextualizes selected records and therefore does not establish native-K/V equivalence. Frozen-trajectory screening also cannot replace autonomous evaluation. SWE-bench coding tasks may not represent research, browser, or multimodal agents. Semantic record labels may depend on heuristics during the first implementation.

These limitations are deliberate. This paper isolates logical memory selection before introducing engine-specific physical state. The current autonomous controls also show that fixed temperature and seed fields do not eliminate trajectory variation or submission failures in the tested endpoint; repeated FULL controls and separately labelled solution-state diagnostics are needed before attributing success or failure to a memory policy. Auxiliary workspace grades are post-hoc diagnostics and never replace the primary agent submission.

15 Conclusion

Agent memory selection is a distinct problem from transformer cache reuse. A serving engine may perfectly preserve selected native K/V while an agent policy selects the wrong history, and a good logical policy may be obscured by an incorrect engine implementation. We therefore study memory selection first, using an engine-independent record interface and a minimal software-engineering agent. The resulting policy frontier should provide a semantic contract for later native-memory runtimes: engines realize a frozen memory plan; they do not decide what the agent ought to remember. The first autonomous evidence further shows that this frontier cannot be estimated from a single control trajectory: the primary FULL submission resolves only one of two executions on each of two task identities, even though auxiliary grading shows both FULL-B workspaces contain resolving patches. The second task therefore fails the gate for heuristic evaluation. H3 remains negative under both primary and auxiliary outcomes on the first task.

A Artifact Schema

Each request/run should persist:

- task ID and repository revision;
- mini-swe-agent revision;
- model/tokenizer revision;
- generation settings;
- canonical trajectory digest;
- typed record list and causal groups;
- selector name/revision;
- materializer name/revision;

- requested/realized budget;
- selected record IDs/spans;
- selected token count;
- next model response/action;
- primary autonomous submission result;
- separately labelled auxiliary final-workspace result and extraction availability;
- reread/retest/recovery counters;
- failure taxonomy labels.

B Initial Failure Taxonomy

```
MEMORY_TASK_LOSS
MEMORY_SOURCE_LOSS
MEMORY_MUTATION_LOSS
MEMORY_VERIFICATION_LOSS
MEMORY_PROGRESS_LOSS
MEMORY_CAUSAL_PAIR_BREAK
MEMORY_STALE_HYPOTHESIS
MEMORY_REDUNDANT_REREAD
MEMORY_REDUNDANT_TEST
MEMORY_WRONG_FILE
MEMORY_REPEAT_FAILED_EDIT
MEMORY_FORMAT_REGRESSION
MEMORY_RECOVERY_SUCCESS
MEMORY_RECOVERY_FAILURE
```